



Aula 13: Login com JWT

⚙ Status Não iniciada

Aula 13 — Login com JWT

Objetivo da aula

Nesta aula, você vai criar o login da API.

Ao final, você vai saber:

- ✓ criar rota de login
- ✓ buscar usuário pelo email
- ✓ comparar senha com bcrypt
- ✓ entender o que é JWT
- ✓ gerar token de autenticação
- ✓ usar `JWT_SECRET` no `.env`
- ✓ testar login no Postman

1. O que vamos construir?

Na aula anterior, você criou o cadastro de usuário.

Agora, você vai criar a rota de login:

```
POST /login
```

Ela vai receber:

```
{
  "email": "ana@email.com",
  "senha": "123456"
}
```

Se estiver tudo certo, a API vai responder com um token:

```
{
  "mensagem": "Login realizado com sucesso.",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..\"
}
```

Esse token será usado nas próximas aulas para acessar rotas protegidas.

2. Como funciona o login? 🧠

O fluxo do login é assim:

1. Usuário envia email e senha
2. API busca o usuário pelo email
3. API verifica se a senha está correta
4. Se estiver correto, a API gera um token
5. O usuário usa esse token para acessar rotas protegidas

Exemplo prático:

```
Usuário quer acessar o perfil
↓
Faz login
↓
Recebe token
↓
```

Envia token nas próximas requisições



API libera acesso

3. O que é JWT?

JWT significa:

JSON Web Token

Na prática, é um token usado para identificar um usuário logado.

Pense nele como uma pulseira de evento.

Você entra no evento, recebe uma pulseira e pode acessar áreas permitidas.

No sistema, funciona parecido:

Usuário faz login



API gera token



Usuário envia token nas próximas requisições



API verifica se o token é válido

4. Instalando o jsonwebtoken

No terminal, instale:

```
npm install jsonwebtoken
```

Essa biblioteca será usada para gerar e validar tokens JWT.

5. Configurando o JWT_SECRET

Abra o arquivo:

```
.env
```

Adicione:

```
JWT_SECRET=sua_chave_secreta_aqui
```

Exemplo:

```
PORT=3000  
MONGO_URL=sua_string_de_conexao  
JWT_SECRET=minha_chave_super_secreta
```

O `JWT_SECRET` é usado para assinar o token.

⚠ Não envie seu `.env` para o GitHub.

6. Por que usar JWT_SECRET?

O `JWT_SECRET` é uma chave secreta da sua aplicação.

Ele serve para garantir que o token foi criado pela sua API.

Exemplo:

```
jwt.sign(dados, process.env.JWT_SECRET);
```

Se alguém tentar criar um token falso sem conhecer o `JWT_SECRET`, a API não deve aceitar.

Por isso, essa informação precisa ficar protegida.

7. Ajustando o service de usuário

Na aula anterior, você criou:

```
src/services/userService.js
```

Confira se ele possui a função para buscar usuário pelo email:

```

const User = require("../models/User");

async function buscarUsuarioPorEmail(email) {
  const usuario = await User.findOne({ email });

  return usuario;
}

async function criarUsuario(dados) {
  const novoUsuario = await User.create(dados);

  return novoUsuario;
}

module.exports = {
  buscarUsuarioPorEmail,
  criarUsuario
};

```

Essa função será usada no login.

8. Criando o controller de autenticação 🎮

Dentro da pasta:

```
src/controllers
```

Crie o arquivo:

```
authController.js
```

Adicione:

```

const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const userService = require("../services/userService");

```

```

async function login(req, res, next) {
  try {
    const { email, senha } = req.body;

    if (!email || !senha) {
      return res.status(400).json({
        erro: "Email e senha são obrigatórios."
      });
    }

    const usuario = await userService.buscarUsuarioPorEmail(
      email);

    if (!usuario) {
      return res.status(401).json({
        erro: "Email ou senha inválidos."
      });
    }

    const senhaCorreta = await bcrypt.compare(senha, usuario.senha);

    if (!senhaCorreta) {
      return res.status(401).json({
        erro: "Email ou senha inválidos."
      });
    }

    const token = jwt.sign(
      {
        id: usuario._id,
        nome: usuario.nome,
        email: usuario.email,
        tipo: usuario.tipo
      },
      process.env.JWT_SECRET,
      {

```

```

        expiresIn: "1d"
      }
    );

    res.json({
      mensagem: "Login realizado com sucesso.",
      token
    });
  } catch (error) {
    next(error);
  }
}

module.exports = {
  login
};

```

9. Entendendo o controller

Recebendo email e senha

```
const { email, senha } = req.body;
```

A API espera receber:

```

{
  "email": "ana@email.com",
  "senha": "123456"
}

```

Validando campos obrigatórios

```

if (!email || !senha) {
  return res.status(400).json({
    erro: "Email e senha são obrigatórios."
  });
}

```

```
});  
}
```

Se faltar email ou senha, retornamos erro `400`.

Buscando usuário pelo email

```
const usuario = await userService.buscarUsuarioPorEmail(email);
```

Se o email não existir, retornamos erro:

```
return res.status(401).json({  
  erro: "Email ou senha inválidos."  
});
```

Usamos a mesma mensagem para email errado e senha errada.

Isso é mais seguro.

10. Comparando senha com bcrypt

Na aula anterior, a senha foi salva criptografada.

Por isso, não podemos comparar assim:

```
senha === usuario.senha
```

Isso não funciona, porque `usuario.senha` está criptografada.

O correto é usar:

```
const senhaCorreta = await bcrypt.compare(senha, usuario.senha);
```

O `bcrypt.compare()` compara a senha digitada com a senha criptografada salva no banco.

Se estiver correta, retorna `true`.

Se estiver errada, retorna `false`.

11. Gerando o token JWT

Depois que email e senha estão corretos, geramos o token:

```
const token = jwt.sign(  
  {  
    id: usuario._id,  
    nome: usuario.nome,  
    email: usuario.email,  
    tipo: usuario.tipo  
  },  
  process.env.JWT_SECRET,  
  {  
    expiresIn: "1d"  
  }  
);
```

O primeiro objeto contém dados do usuário.

Esses dados são chamados de payload.

12. O que é payload?

Payload é a informação que fica dentro do token.

Neste caso:

```
{  
  id: usuario._id,  
  nome: usuario.nome,  
  email: usuario.email,  
  tipo: usuario.tipo  
}
```

Esses dados ajudam a API a identificar quem está logado.

 Nunca coloque senha dentro do token.

Errado:

```
{
  id: usuario._id,
  senha: usuario.senha
}
```

Certo:

```
{
  id: usuario._id,
  email: usuario.email,
  tipo: usuario.tipo
}
```

13. Expiração do token

Aqui definimos que o token expira em 1 dia:

```
{
  expiresIn: "1d"
}
```

Isso significa que, depois desse tempo, o usuário precisa fazer login novamente.

Outros exemplos:

```
expiresIn: "1h"
expiresIn: "7d"
expiresIn: "30d"
```

Para estudo, `1d` está ótimo.

14. Criando as rotas de autenticação

Dentro da pasta:

```
src/routes
```

Crie o arquivo:

```
authRoutes.js
```

Adicione:

```
const express = require("express");

const authController = require("../controllers/authController");

const router = express.Router();

router.post("/login", authController.login);

module.exports = router;
```

15. Registrando a rota no server.js 🚀

Abra:

```
src/server.js
```

Importe a rota:

```
const authRoutes = require("../routes/authRoutes");
```

Depois registre:

```
app.use(authRoutes);
```

Seu `server.js` deve ficar parecido com:

```
require("dotenv").config();

const express = require("express");

const connectDatabase = require("./config/database");
const productRoutes = require("./routes/productRoutes");
const userRoutes = require("./routes/userRoutes");
const authRoutes = require("./routes/authRoutes");
const logger = require("./middlewares/loggerMiddleware");
const errorHandler = require("./middlewares/errorMiddleware");

const app = express();

connectDatabase();

app.use(express.json());
app.use(logger);

app.use(productRoutes);
app.use(userRoutes);
app.use(authRoutes);

app.use((req, res) => {
  res.status(404).json({
    erro: "Rota não encontrada."
  });
});

app.use(errorHandler);

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```


Use:

Body:

Depois teste o login:

Body:

Resposta esperada:

17. Testando senha errada ❌

Envie:

```
{
  "email": "ana@email.com",
  "senha": "senhaerrada"
}
```

Resposta esperada:

```
{
  "erro": "Email ou senha inválidos."
}
```

Status esperado:

401 Unauthorized

18. Testando email inexistente ❌

Envie:

```
{
  "email": "naoexiste@email.com",
  "senha": "123456"
}
```

Resposta esperada:

```
{
  "erro": "Email ou senha inválidos."
}
```

Usar a mesma mensagem para email errado e senha errada é uma boa prática. Assim, você não informa para alguém mal-intencionado se aquele email existe ou não.

19. Testando campos obrigatórios ❌

Envie:

```
{  
  "email": "ana@email.com"  
}
```

Resposta esperada:

```
{  
  "erro": "Email e senha são obrigatórios."  
}
```

Status esperado:

400 Bad Request

20. Como visualizar o token?

O token JWT parece uma string grande:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Ele normalmente tem 3 partes separadas por ponto:

```
header.payload.signature
```

Exemplo:

```
xxxxx.yyyyy.zzzzz
```

Você não precisa decorar isso agora.

Para esta etapa, o mais importante é entender:

```
Login correto gera token.
```

Token será usado para acessar rotas protegidas.

21. O que vem na próxima aula?

Na próxima aula, você vai criar rotas protegidas.

Hoje você gerou o token.

Na próxima aula, você vai aprender a enviar esse token para a API e verificar se o usuário pode acessar determinada rota.

Exemplo:

```
GET /perfil
```

Essa rota só deve funcionar para quem estiver logado.

22. Código final da aula

.env

```
PORT=3000
MONGO_URL=sua_string_de_conexao
JWT_SECRET=sua_chave_secreta
```

src/controllers/authController.js

```
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
const userService = require("../services/userService");

async function login(req, res, next) {
  try {
    const { email, senha } = req.body;

    if (!email || !senha) {
```

```

        return res.status(400).json({
            erro: "Email e senha são obrigatórios."
        });
    }

    const usuario = await userService.buscarUsuarioPorEmail(
        email);

    if (!usuario) {
        return res.status(401).json({
            erro: "Email ou senha inválidos."
        });
    }

    const senhaCorreta = await bcrypt.compare(senha, usuario.senha);

    if (!senhaCorreta) {
        return res.status(401).json({
            erro: "Email ou senha inválidos."
        });
    }

    const token = jwt.sign(
        {
            id: usuario._id,
            nome: usuario.nome,
            email: usuario.email,
            tipo: usuario.tipo
        },
        process.env.JWT_SECRET,
        {
            expiresIn: "1d"
        }
    );

    res.json({
        mensagem: "Login realizado com sucesso.",
    });

```

```

        token
    });
} catch (error) {
    next(error);
}
}

module.exports = {
    login
};

```

src/routes/authRoutes.js

```

const express = require("express");

const authController = require("../controllers/authController");

const router = express.Router();

router.post("/login", authController.login);

module.exports = router;

```

src/server.js

```

require("dotenv").config();

const express = require("express");

const connectDatabase = require("../config/database");
const productRoutes = require("../routes/productRoutes");
const userRoutes = require("../routes/userRoutes");
const authRoutes = require("../routes/authRoutes");
const logger = require("../middlewares/loggerMiddleware");
const errorHandler = require("../middlewares/errorMiddleware");

```

```

e");

const app = express();

connectDatabase();

app.use(express.json());
app.use(logger);

app.use(productRoutes);
app.use(userRoutes);
app.use(authRoutes);

app.use((req, res) => {
  res.status(404).json({
    erro: "Rota não encontrada."
  });
});

app.use(errorHandler);

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});

```

23. Resumo da aula

Nesta aula, você aprendeu:

- ✓ login usa email e senha
- ✓ a API deve buscar o usuário pelo email
- ✓ senha criptografada não pode ser comparada com `===`
- ✓ `bcrypt.compare()` compara senha digitada com senha criptografada
- ✓ JWT gera um token de autenticação

- ✓ `JWT_SECRET` deve ficar no `.env`
- ✓ o token não deve conter senha
- ✓ erro de login deve usar status `401`
- ✓ login correto retorna um token

Exercícios

Exercício 1 — Criar login de usuário

Crie a rota:

```
POST /login
```

Ela deve receber:

```
{
  "email": "ana@email.com",
  "senha": "123456"
}
```

Regras:

- email é obrigatório
- senha é obrigatória
- se o usuário não existir, retornar `401`
- se a senha estiver errada, retornar `401`
- se estiver tudo certo, retornar token JWT

Exercício 2 — Melhorar resposta do login

Na resposta do login, além do token, retorne os dados básicos do usuário.

Exemplo:

```
{
  "mensagem": "Login realizado com sucesso.",

```

```
"token": "token_aqui",
"usuario": {
  "id": "665f1f7a3c9f4b0012a8e999",
  "nome": "Ana",
  "email": "ana@email.com",
  "tipo": "cliente"
}
```

Atenção:

Não retorne a senha.

Exercício 3 — Testar erros no Postman

Teste no Postman os seguintes cenários:

1. Login correto
2. Senha errada
3. Email inexistente
4. Sem email
5. Sem senha

Para cada caso, anote:

URL usada
Body enviado
Status retornado
Resposta da API